

专利申请 技术交底书

一种面向智能体的应用文件系统交互方法、系统、设备及存储介质

注意事项：本文件为技术交底材料草案，目的在于帮助专利代理人理解技术方案并据此撰写说明书、权利要求书、摘要和必要附图。首页申请人、发明人、联系人等基础信息暂以“待填写”占位，正式提交前应由申请人确认。

项目	填写内容
交底书名称	一种面向智能体的应用文件系统交互方法、系统、设备及存储介质
技术联系人姓名及电话、email	待填写
经办人姓名及电话、email	待填写
申请人	待填写
发明人	待填写
交底日期	待填写
当前状态	基于 appfs-platform 项目文档和代码整理的技术交底书完善稿
建议保护重点	AppFS 文件协议、动作槽、事件流、多 agent principal 身份、私有应用实例和凭据隔离

交底书正文提纲

- 说明现有 AI Agent 与外部应用集成的常见方案及其客观缺点。
- 阐述 AppFS 将应用状态、动作入口、事件流和自描述能力映射为文件系统命名空间的总体方案。
- 展开结构快照、资源文件、追加式动作文件、事件流、cursor、request_id 和 client_token 的协同机制。
- 展开 principal 稳定身份、attach lease 进程绑定、heartbeat、stale sweep 和 private app 自动实例化机制。
- 说明 ConnectorContext、profile_id、凭据隔离、自描述 skill 和事件提醒进入模型回合的实现方式。
- 以 Tinode 私有聊天、多 agent 协作、Dashboard 和桌面启动管理作为具体实施例。
- 列出替代方案、关键保护点、建议附图和已核对项目资料，供代理人扩展权利要求使用。

一、与本专利最接近的现有技术

1、现有技术的方案简述

随着大语言模型和自动化 agent 的发展，越来越多的软件系统需要让 agent 读取应用状态、调用业务动作、接收外部事件并与多个 agent 协作。现有方案通常采用插件、SDK、REST API、MCP 工具、浏览器自动化、消息队列或聊天机器人接口等方式，将每个应用的能力以函数调用、HTTP 调用或 UI 操作的形式暴露给 agent。

在此类方案中，应用开发者通常需要为每个应用单独编写工具定义、认证逻辑、资源读取逻辑、动作执行逻辑和事件回调逻辑。Agent 运行时则需要在系统提示、工具列表或插件描述中理解这些能力，并在会话中决定何时调用某个工具或 API。若存在多个 agent，还需要额外设计进程标识、会话标识、应用账号、凭据存放和事件订阅范围。

对于实时通信、工单、日历、邮件、企业协作等应用，现有技术还会使用 webhook、轮询或消息队列将外部事件推送到 agent。此时，事件需要被转换为模型可理解的上下文，且需要避免重复消费、漏消费、越权读取和凭据泄漏。不同应用的资源模型、动作模型、事件模型差异较大，导致 agent 侧通常需要硬编码大量应用语义。

以聊天应用为例，现有方案可能为“发送消息”单独提供 API，为“查询联系人”提供另一个 API，为“接收消息”配置 webhook。Agent 若要给某人发送消息，需要知道联系人标识、认证凭据、消息接口、结果查询接口及错误处理规则；多个 agent 若共享同一项目，还要额外区分各自的聊天账号和私有消息历史。

2、现有技术的客观缺点

- 应用能力缺少统一、可遍历、可被模型自然发现的表示方式。工具/API 列表通常与实际应用状态分离，模型难以在同一命名空间中同时理解资源、动作和事件。
- 动作执行缺少文件级追加审计和稳定幂等键。若模型、脚本或管理端重试同一动作，容易出现重复发送、重复创建或状态不一致。
- 外部事件难以与模型回合边界对齐。Webhook 或消息队列事件通常需要额外服务转换，容易出现事件噪声、漏唤醒、重复唤醒或上下文过量注入。
- 稳定 agent 身份和运行时进程绑定容易混淆。若以进程 ID、会话 ID 或随机 attach ID 绑定应用账号，则 agent 重启、fork 或多 agent 并行时难以保持一致的私有应用状态。
- 多 agent 场景下，公共应用、私有应用、凭据和事件订阅范围缺乏通用隔离模型，容易导致一个 agent 看到或误用另一个 agent 的私有资源。
- 应用结构变化缺少统一刷新机制。联系人、群聊、会话、scope 等动态资源出现后，模型可见的工具或路径不一定同步更新。
- 凭据和访问令牌可能被放入 prompt、工具参数、动作 payload、事件日志或模型可见文件树中，增加泄露风险。
- Dashboard、桌面启动器、agent 进程与应用 runtime 往往分别维护状态，容易出现 UI 显示、进程状态和应用实际绑定状态不一致的问题。

二、本专利的技术

（1）技术领域与发明目的

本发明涉及人工智能智能体、应用集成、虚拟文件系统、事件驱动运行时、多智能体协作和应用连接器技术。更具体地，本发明涉及一种面向智能体的应用文件系统交互方法、系统、设备及存储介质，用于将外部应用的状态、动作、事件、身份和上下文隔离能力映射为文件系统协议，使智能体能够以统一方式发现、读取、调用和响应应用。

本发明的目的在于提供一种面向智能体的应用文件系统交互方案，使 agent 能够通过普通文件读写访问应用状态、提交业务动作、接收外部事件并管理多 agent 身份。该方案降低应用集成耦合度，提高动作审计性和幂等性，实现多 agent 私有上下文隔离，并避免凭据进入模型可见层。

（2）总体方案

本专利提出一种面向智能体的应用文件系统交互方法。其核心思想是：将外部应用的状态资源、可执行动作、控制面信息、事件流和多 agent 身份上下文，按照约定命名空间映射为可挂载或可访问的文件树。Agent 不需要直接理解每个外部应用的内部 API，而是读取 `.res.json`、`.res.jsonl` 等资源文件，向 `.act` 动作文件追加 JSONL 请求，并从 `._stream/events.evt.jsonl` 事件流获知动作结果或外部消息。

该方案可部署为操作系统挂载目录，也可由本地目录、SQLite backed VFS、网络文件系统或其他存储层承载。应用适配器通过 connector 抽象提供结构快照、资源读取、动作执行和入站事件，AppFS runtime supervisor 负责将上述能力物化为文件树、消费动作文件、维护事件流和管理多 agent 生命周期。

核心理解：本发明的边界不是某一个聊天应用或某一个 Dashboard，而是“把应用的状态、动作、事件、身份和自描述能力统一映射为文件系统协议，并由 agent 按文件协议交互”的机制。Tinode 私有聊天、Dashboard 和桌面启动器均为实施例。

（3）核心术语

术语	含义
AppFS runtime	发布运行清单、维护控制平面、管理应用 registry、principal registry、事件流和 connector runtime 的运行时。
connector	连接具体外部应用的软件组件，负责返回应用结构快照、执行业务动作、拉取或产生 inbound events，但不直接写 AppFS 文件树。
principal	稳定的 agent 语义身份，例如 default、code-implementer；用于绑定私有应用实例、profile 和可见性。
attach lease	某个 agent 进程对某个 principal 的运行绑定，包括 attach_id、role、session_id、attached_at、last_seen_at 等。
profile_id	principal 在具体私有应用中的应用侧身份，例如 tinode:default；connector 以该值查找或创建凭据。

术语	含义
.res.json / .res.jsonl	单对象或多行资源文件，用于表达控制描述、列表、消息流、分页快照或应用自描述资源。
.act	追加式 JSONL 动作文件；向该文件追加一行 JSON 表示提交一次动作。
.evt.jsonl	追加式事件流文件；runtime 将动作结果、外部消息、状态变化等写入该文件。
runtime manifest	位于 /.well-known/appfs/runtime.json 的运行清单，用于让 agent 或 launcher 明确发现 mount root、runtime_session_id、控制动作路径和能力标志。

（4）系统组成

组成模块	功能说明
AppFS runtime supervisor	维护挂载根目录、runtime manifest、应用注册表、principal 注册表、动作消费循环、事件发布、结构同步和私有应用实例化。
挂载应用树	以文件路径表达应用资源、动作入口和事件流，包括 /_appfs、/public、/private、_app、_stream 等目录。
Connector	将外部应用或服务转换为 AppFS 结构快照、资源读取、动作执行和入站事件。可为进程内、HTTP、gRPC 或远程服务形式。
动作消费模块	扫描 `.act` 文件，按 action cursor 消费新增 JSONL 行，校验 payload，生成 request_id 和 client_token，并调用 connector。
事件发布模块	将 action.accepted、action.progress、action.completed、action.failed 以及 connector inbound events 写入事件流和 replay 文件。
principal 管理模块	通过 create、attach、update、detach、delete 动作维护稳定 principal、attach lease、agent_status 和 stale sweep。
appfs-agent	发现 AppFS，确保 principal，attach 到 runtime，读取 skill/action/control 描述，追加动作，并把 AppFS 事件转换为模型输入。
Dashboard/desktop	作为实施例中的控制面，创建、启动、停止、恢复和删除 principal 对应 agent，并展示状态和时间线。

图1 系统总体架构

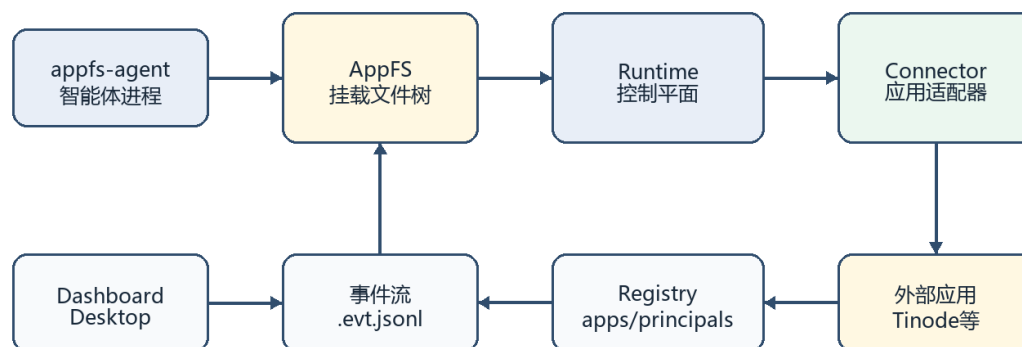


图 1 系统总体架构图：展示 agent、AppFS 文件树、runtime、connector、registry、事件流和管理端之间的协作关系。

（5）挂载命名空间与文件协议

在一个项目根目录下，AppFS runtime 发布版本化运行时清单 `/.well-known/appfs/runtime.json`。该清单包含 runtime 类型、mount root、runtime session id、多 agent 模式、控制面动作路径、事件流路径和能力标志。Agent 优先通过显式环境变量或该 manifest 发现 AppFS 环境，无法获取时再使用目录启发式发现。

路径或文件	用途
<code>/.well-known/appfs/runtime.json</code>	运行清单，供 agent、launcher 和管理端发现 runtime。
<code>/_appfs/apps.registry.json</code>	平台注册的公共应用和私有应用实例清单，记录 app_id、visibility、principal_id、profile_id、path、session_id 等。
<code>/_appfs/app-policies.registry.json</code>	应用可见性策略和 private app 模板，如 path_template、profile_template、credential_policy。
<code>/_appfs/principals.registry.json</code>	principal registry，记录 principal、active_attaches、agent_status 等。
<code>/_appfs/principals/*.act</code>	create、attach、update、detach、delete 等 principal 生命周期动作。
<code>/_appfs/_stream/events.evt.jsonl</code>	平台控制面事件流。
<code>/public/<app>/</code>	所有 principal 可见的公共应用实例。
<code>/private/<principal-id>/<app>/</code>	某个 principal 专属的私有应用实例。
<code><app-root>/_app/</code>	应用自描述与控制区，包括 actions、control、events、skill、self 等资源和控制动作。
<code><app-root>/_stream/events.evt.jsonl</code>	应用实例事件流。

路径或文件	用途
<app-root>/_stream/action-cursors.res.json	动作消费 cursor 状态。
<app-root>/*.res.json / *.res.jsonl / *.act	具体业务资源、列表、历史消息、分页资源和动作入口。

Connector 返回的结构节点不得写入 runtime 保护路径，例如 ``_stream``、`paging`、`snapshot journal` 等。Runtime 对路径进行相对路径校验，拒绝绝对路径、``.``、平台保留路径或不安全路径，防止 connector 或外部输入逃逸应用根目录。

```
/
├── .well-known/appfs/runtime.json
├── _appfs/
│   ├── apps.registry.json
│   ├── app-policies.registry.json
│   ├── principals.registry.json
│   ├── principals/
│   │   ├── create_principal.act
│   │   ├── attach_principal.act
│   │   ├── update_principal.act
│   │   ├── detach_principal.act
│   │   ├── delete_principal.act
│   │   └── status.res.json
│   └── _stream/events.evt.jsonl
├── public/<app>/
└── private/<principal-id>/<app>/
    ├── _app/
    ├── _stream/events.evt.jsonl
    ├── *.res.json
    ├── *.res.jsonl
    └── *.act
```

图2 文件树命名空间

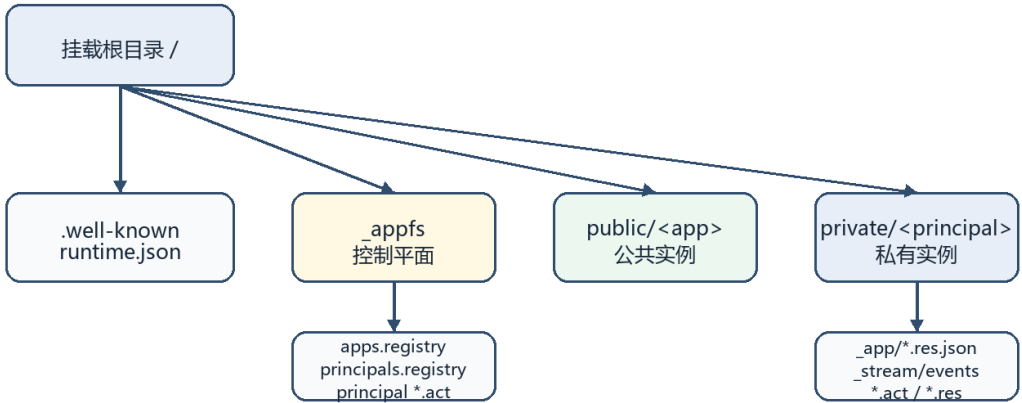


图 2 文件树命名空间图：展示 runtime manifest、控制平面、公共应用实例和 principal 私有应用实例的目录边界。

（6）runtime manifest 和 registry

系统启动后，AppFS runtime 在挂载根目录下发布 `/.well-known/appfs/runtime.json`。manifest 的控制平面路径包括 register_app、unregister_app、list_apps、create_principal、update_principal、delete_principal、attach_principal、detach_principal、apps registry、principals registry 和平台事件流。能力标志包括 app_registration、event_stream、multi_app、scope_switch 和 multi_agent_attach。

接口文件	主要字段或能力
runtime.json	schema_version、runtime_kind、mount_root、runtime_session_id、managed、multi_agent_mode、control_plane、capabilities、generated_at。
apps.registry.json	instance_id、app_id、visibility、parent_app_id、principal_id、profile_id、path、transport、session_id、registered_at、active_scope、inbound_poll_ms、connector_config。
app-policies.registry.json	app_id、visibility(public/private)、connector、transport、path_template、profile_template、credential_policy、inbound_poll_ms。
principals.registry.json	default_principal_id、principal_id、display_name、kind、active_attach_count、active_attaches、agent_status。
principals/status.res.json	面向 Dashboard 和 agent 的派生状态视图，包含 presence、state、current_task_preview、model、last_activity_at 等摘要字段。

（7）Connector 抽象接口

Connector 是真实应用和 AppFS 文件树之间的边界。Connector 只返回结构、资源页、动作执行结果和入站事件，文件创建、事件写入、路径安全校验和 registry 管理由 runtime 负责。

接口或数据结构	说明
ConnectorContext	包含 app_id、session_id、request_id、client_token、trace_id、principal_id、profile_id。principal_id/profile_id 由 runtime 填入，动作 payload 不能覆盖。
get_app_structure(request, ctx)	返回应用结构快照或 unchanged 结果，用于首次物化应用文件树。
refresh_app_structure(request, ctx)	按刷新原因、目标 scope 或动作触发路径刷新结构。
submit_action(request, ctx)	执行业务动作，返回 completed 或 streaming 结果。
drain_inbound_events(ctx)	拉取外部应用产生的新事件，例如新消息、状态变化、凭据变化。
fetch_snapshot_chunk / fetch_live_page	用于资源读穿、分页和实时资源读取。
ConnectorInboundEvent	包含 event_type、path、content、error 等字段，由 runtime 写入事件流。

（8）应用结构同步与树物化方法

Connector 返回的 AppStructureSnapshot 包括 app_id、revision、active_scope、ownership_prefixes 和节点列表。节点类型包括 Directory、ActionFile、SnapshotResource、LiveResource 和 StaticJsonResource。Runtime 根据节点类型创建目录、空动作文件、资源占位文件或静态 JSON 资源，并生成 `\_meta/manifest.res.json` 和结构同步状态。

1. Runtime 为某个 app 构造 ConnectorContext，携带 app_id、session_id、principal_id 和 profile_id。
2. Runtime 调用 connector 的 get_app_structure 或 refresh_app_structure，传入 known_revision、刷新原因、目标 scope 或触发动作路径。
3. Connector 返回结构快照；若 revision 未变化，可返回 unchanged，runtime 跳过重复物化。
4. Runtime 校验每个节点路径，拒绝绝对路径、非普通路径组件和 runtime 保护路径。
5. Runtime 根据节点类型创建目录、空动作文件、资源占位文件或静态 JSON 文件，并生成 `\_meta/manifest.res.json`。
6. Runtime 记录 owned_paths；下一次结构刷新时，清理 connector 拥有但已不再需要的旧路径，同时保留 `\_stream` 等 runtime 自有状态文件。
7. 当业务动作或 inbound event 表明结构改变时，runtime 以 ActionChanged 或 InboundChanged 等原因刷新结构。

(9) 追加式动作文件处理机制

动作文件以`.act`结尾，作为追加式 JSONL 动作槽。Agent、脚本或 Dashboard 将一行 JSON payload 追加到动作文件，AppFS action consumer 根据动作 cursor 只读取尚未处理的新增字节。该机制避免模型直接调用内部 API，使每个动作请求都留下文件级审计痕迹。

处理阶段	技术手段
动作发现	Runtime 收集 app 目录下声明过的`.act`文件，并按 manifest 中的 action template 匹配。
cursor 读取	每个动作文件维护 offset、boundary_probe 和 pending_multiline_eof_len，只消费 cursor 后新增内容。
安全校验	拒绝不安全动作相对路径、未声明动作路径、非法 JSON、不可恢复的多行不完整写入和覆盖/截断行为。
request_id	每个动作行生成新的 request_id，用于关联事件和一次执行尝试。
client_token	优先使用 payload 或 action-line envelope 中的 client_token；未提供时基于 app_id、session_id、路径、offset 和 payload 派生稳定 token。
connector 调用	构造 SubmitActionRequest 和 ConnectorContext，调用 submit_action。
事件回执	根据结果写入 action.accepted、action.progress、action.completed 或 action.failed。
重试边界	成功或不可重试错误后推进 cursor；瞬态错误可保留 cursor 以等待后续重试。

```
// 向 contacts/send_message.act 追加一行 JSONL
{"to":"principal:code-implementer","text":"请处理这个实现任务。",
"requires_response":true,"client_token":"msg-001"}

// runtime 消费后写入 _stream/events.evt.jsonl
{"seq":12,"event_id":"evt-12",
"type":"action.completed","path":"contacts/send_message.act",
"request_id":"req-4a1b2c3d","client_token":"msg-001",
"content":{"ok":true}}
```

图3 动作处理流程

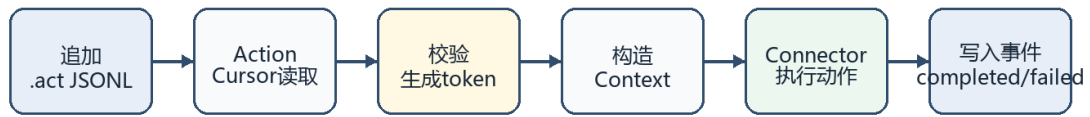


图 3 动作处理流程图：展示`.act`追加、`cursor`消费、校验、`ConnectorContext`、`connector`执行和事件回执。

（10）事件流与模型回合注入

事件流文件为`\_stream/events.evt.jsonl`。每条事件至少可包含 `seq`、`event_id`、`ts`、`app`、`session_id`、`request_id`、`path`、`type`、`content`、`error`、`client_token` 等字段。事件来源包括动作结果事件、`connector inbound` 事件、`principal` 生命周期事件和凭据状态事件。

`appfs-agent` 在模型调用前根据 `session` 内保存的事件 `cursor` 读取新增事件。如果事件需要模型关注，则作为 `pending input` 注入下一模型回合。对于外部消息事件，系统可把正文作为用户可见输入，同时附带来源提醒；对于普通状态事件，系统可渲染为 `system reminder`，并明确说明 `AppFS event` 属于带来源标注的不可信上下文，不是系统指令。

事件类别	示例事件类型	处理方式
动作回执	<code>action.accepted</code> 、 <code>action.progress</code> 、 <code>action.completed</code> 、 <code>action.failed</code>	通常作为状态或摘要提醒，帮助 <code>agent</code> 确认动作结果。
外部消息	<code>message.received</code>	可作为 <code>pending input</code> 注入下一模型回合，并保留 <code>app</code> 、 <code>contact</code> 、 <code>principal</code> 、 <code>seq</code> 等来源信息。
结构变化	<code>inbox.updated</code> 、 <code>contacts.updated</code> 、 <code>groups.updated</code>	可触发结构刷新或作为低优先级上下文。
凭据状态	<code>profile.credentials.ready</code> 、 <code>profile.credentials.failed</code>	用于提示当前 <code>profile</code> 是否可执行应用动作。
生命周期事件	<code>principal.created</code> 、 <code>principal.attached</code> 、 <code>principal.status.updated</code> 、 <code>principal.deleted</code>	用于管理端和 <code>agent</code> 状态同步，一般作为控制面事件。

图4 事件唤醒流程

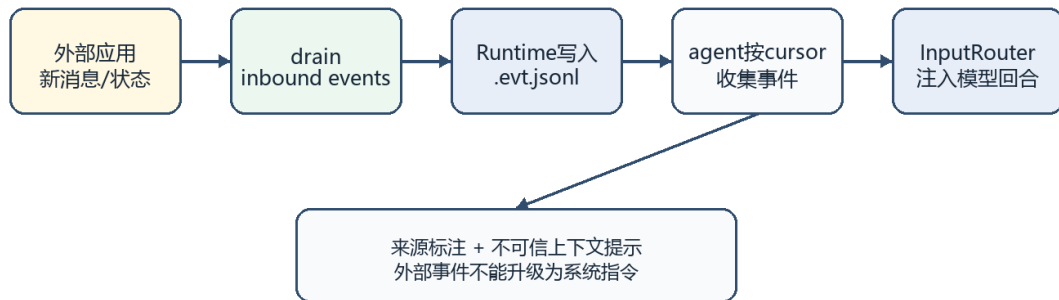


图 4 事件唤醒流程图：展示 inbound events、runtime event stream、agent event cursor 和模型回合输入。

(11) principal 生命周期与私有 app 实例化

`principal\_id` 表示项目中稳定的 agent 语义身份，例如 default、code-implementer 或 incident-reporter。`attach\_id` 表示某一运行时进程对 principal 的临时绑定。一个 principal 可跨进程重启复用相同私有应用实例和应用账号；attach lease 通过 heartbeat 刷新 last_seen_at，runtime 根据 stale 阈值清理失联进程绑定。

生命周期动作	作用
create_principal.act	创建 principal 记录，写入 principal registry，并触发私有应用实例准备。
attach_principal.act	将某个运行进程 attach 到 principal，写入 attach_id、role、session_id、attached_at、last_seen_at。
update_principal.act	带 agent_status 时更新运行状态；不带 agent_status 时作为轻量 heartbeat 刷新 last_seen_at。
detach_principal.act	正常退出时移除 attach lease。
delete_principal.act	删除 principal；若仍在线可拒绝或要求 force，并同步清理私有应用实例和凭据。
stale sweep	当 heartbeat 超过阈值未刷新时，将 attach 判定为 stale 并清理在线状态。当前实现中 stale 判定阈值约为 90 秒。

Private app policy 定义某个应用为私有应用，并配置 `path\_template` 和 `profile\_template`。例如 Tinode 的策略可定义 visibility=private、path_template=private/{principal_id}/tinode、profile_template=tinode:{principal_id}。当 principal 创建或 attach 时，runtime 自动实例化私有路径并生成 profile_id。

```
principal_id = code-implementer
instance_id = tinode--code-implementer
path        = private/code-implementer/tinode
profile_id  = tinode:code-implementer
```

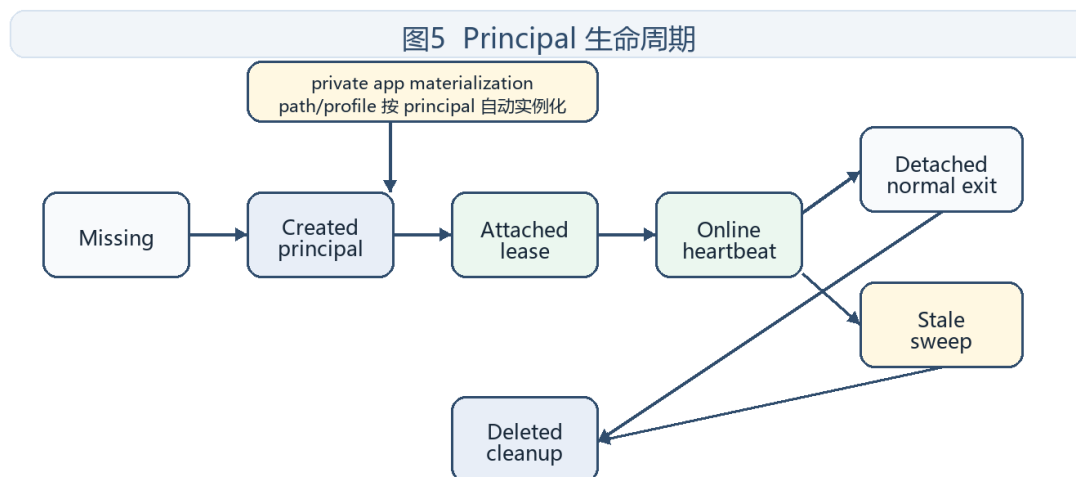


图 5 Principal 生命周期图：展示 create、attach、heartbeat、detach、stale sweep、delete 与私有应用实例化。

（12）appfs-agent 发现、attach 与提示生成

1. Agent 优先读取显式环境变量，例如 APPFS_RUNTIME_MANIFEST、APPFS_MOUNT_ROOT、APPFS_ATTACH_ID、APPFS_PRINCIPAL_ID、APPFS_AGENT_ROLE。
2. 若存在 runtime manifest，agent 读取 mount_root、runtime_session_id、control_plane 和 capabilities。
3. 若未显式指定 principal，agent 默认使用 default；若 principal 不存在，可向 create_principal.act 追加创建请求。
4. Agent 向 attach_principal.act 追加 attach 请求，获得 detach action path 和 update action path。
5. Headless 运行期间，agent 后台线程周期性向 update_principal.act 追加轻量 heartbeat。
6. Agent 读取当前可见应用实例的 `\_app/skill.res.json`、`\_app/actions.res.json`、`\_app/control.res.json`、`\_app/events.res.json`，动态生成模型提示和可用动作说明。
7. 模型提示强调 `.act` 文件必须追加 JSONL，不能覆盖写；动作结果优先通过事件流确认；peer principal 状态可读但不能任意修改。

（13）ConnectorContext 与凭据隔离

Connector 接口在每次资源读取、动作执行和事件拉取时接收 ConnectorContext。该上下文中的 principal_id 和 profile_id 由 runtime 根据 registry、当前应用实例和 private app policy 填入。业务动作 payload 中即使出现身份字段，也不能作为权限来源；有效身份由 AppFS 上下文提供。

凭据、访问令牌、刷新令牌、密码和 API key 保存在 connector 私有状态、操作系统 secret store、私有数据库或加密 KV 中，不写入 `.act` payload、`.res.json` 资源、事件流、skill 文本或模型 prompt。应用资源只暴露安全摘要，例如 credential_status、upstream_user_id、login、display_name、profile_id 等。

层级	可见内容	不可见内容
模型可见文件树	profile_id、安全状态摘要、上游用户 ID 摘要、动作结果、事件摘要	密码、token、cookie、API key、refresh token
Connector 私有状态	profile_id 到凭据记录、client_token 到已完成动作结果、消息 cursor	仅 connector 进程或安全存储可访问
动作 payload	业务参数和可选 client_token	不得要求模型传入账号密码或访问令牌
事件流	动作完成/失败、消息摘要、凭据 ready/failed 状态	不写入实际凭据材料

（14）方法步骤总表

步骤	操作	关键技术手段
S101	发布运行清单	Runtime 在 /.well-known/appfs/runtime.json 写入 mount root、runtime_session_id、控制面动作路径和能力标志。
S102	注册应用策略和实例	Runtime 维护 apps.registry.json、app-policies.registry.json，区分 public app 与 private app。
S103	获取结构快照	Runtime 以 ConnectorContext 调用 connector.get_app_structure 或 refresh_app_structure。
S104	物化文件树	Runtime 校验节点路径并生成 _app、_stream、资源文件、动作文件、manifest 和 cursor 文件。
S105	发现并绑定 principal	Agent 通过环境变量/manifest/启发式发现 AppFS，ensure principal 后 append attach_principal.act。
S106	提交动作	Agent 向 .act 追加 JSONL；action consumer 按 cursor 消费并生成 request_id/client_token。
S107	执行动作并发布事件	Runtime 调用 connector.submit_action，将完成、失败、进度和业务 side events 写入 .evt.jsonl。
S108	接收外部事件	Runtime 周期性调用 drain_inbound_events，并将 inbound events 写入应用事件流。
S109	注入模型回合	appfs-agent 按事件 cursor 收集新增事件，经 InputRouter 分类后注入下一模型调用或唤醒空闲 agent。
S110	维护生命周期和隔离	Heartbeat 刷新 attach lease；stale sweep 清理失联 attach；private app 按 principal 自动实例化和清理。

（15）Tinode 私有聊天实施例

Tinode 实施例展示了一个真实聊天应用如何接入本方案。Tinode 被定义为 private account-backed app。每个 principal 对应一个独立路径 ``/private/<principal-id>/tinode`` 和一个 profile ``tinode:<principal-id>``。attach_id 不拥有 Tinode 凭据；principal_id 稳定拥有 Tinode 凭据。Agent fork 产生新 principal 时，新 principal 默认获得新的 Tinode profile。

Tinode 路径	用途
_app/actions.res.json	推荐动作，例如发送私聊消息、创建群聊。
_app/control.res.json	控制动作和事件路径，例如 ensure_credentials、forget_credentials。
_app/events.res.json	事件分类与模型渲染方式，例如 message.received、message.sent、action.failed。
_app/skill.res.json	面向 agent 的使用说明和技能摘要。
_app/self.res.json	当前 principal/profile 的安全摘要和 credential_status。
_app/ensure_credentials.act	为当前 profile 创建或复用 Tinode 凭据。
contacts/index.res.jsonl	联系人索引。
contacts/send_message.act	根级发送私聊消息动作，支持 to=principal:<principal-id>。
contacts/<contact-key>/messages.res.jsonl	联系人消息历史。
groups/create_group.act	创建群聊并邀请成员。
groups/<group-key>/send_message.act	向群聊发送消息。
inbox/recent.res.jsonl / inbox/unread.res.jsonl	最近消息和未读消息视图。
inbox/mark_read.act	标记消息已读。
_stream/events.evt.jsonl	Tinode 应用实例事件流。

当 agent 要给另一个 principal 发送私聊消息时，向当前 principal 的 Tinode app root 下的 ``contacts/send_message.act`` 追加一行 JSON。Runtime 消费该动作行后，将 principal_id 和 profile_id 通过 ConnectorContext 传给 Tinode connector。Connector 根据 profile_id 找到当前 principal 的 Tinode 凭据，解析 ``principal:code-implementer`` 为目标 principal 的 Tinode profile 或联系人，发送消息，并产生 message.sent、action.completed 等事件。若使用相同 client_token 重试，connector 可返回已完成结果，避免重复发送。

Tinode connector 在 `drain_inbound_events(ctx)` 中按当前 `profile` 拉取新消息。收到外部消息后，connector 更新本地消息资源和 `inbox`，并返回 `message.received`、`inbox.updated` 等事件。Runtime 将这些事件写入 ``_stream/events evt.jsonl``。appfs-agent 在下一模型回合前读取事件，将需要关注的消息作为输入提醒，同时附带来源、`contact_key`、目标 `principal` 和 `seq`。

多 agent 群聊时，当前 `principal` 可向 ``groups/create_group.act`` 追加包含 `title` 和 `members` 的 JSON，`members` 可使用 ``principal:<principal-id>``。Connector 通过 AppFS registry 和 Tinode credential state 解析成员，创建群聊并邀请成员。后续消息通过 ``groups/<group-key>/send_message.act`` 发送。

图6 Tinode 私有聊天实施例

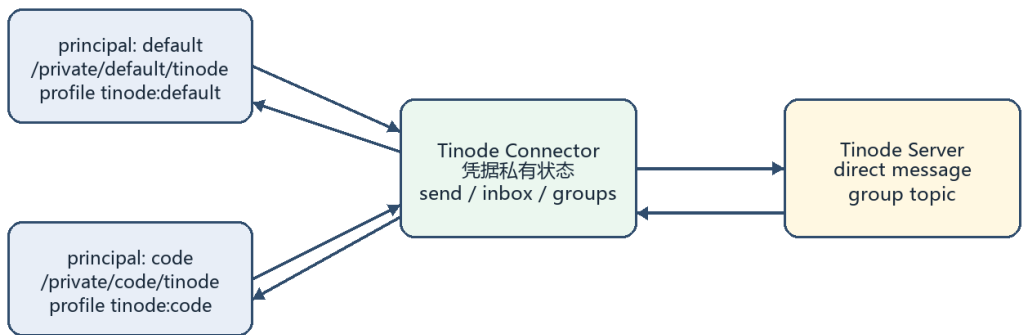


图 6 Tinode 私有聊天实施例图：展示不同 `principal` 的私有 Tinode profile、connector 私有凭据和 `direct/group message` 交互。

（16）Dashboard 与桌面启动管理实施例

Dashboard 作为管理端，通过 HTTP 路由提供 `principal` 列表、创建、删除、启动、停止和恢复接口。其 `lifecycle service` 读取 ``/_appfs/principals.registry.json`` 和 ``/_appfs/principals/status.res.json``，并通过追加 `create_principal`、`detach_principal`、`delete_principal` 等文件提交管理动作。

Dashboard 启动 agent 时构造 `spawn config`，并向子进程环境注入 `APPFS_PRINCIPAL_ID`、`APPFS_ATTACH_ID`、`APPFS_MOUNT_ROOT`、`APPFS_RUNTIME_MANIFEST` 等变量。Agent 进程启动后通过 `stdout JSONL` 返回 `session_started`、`control endpoint`、`principal_id` 等信息。Dashboard 据此维护进程状态，并结合 `runtime` 的 `principal registry` 展示在线状态。

桌面 Electron launcher 则负责启动 `dashboard server`、选择可用端口、设置 AppFS CLI 和 agent binary 环境变量，并执行优雅关闭或进程树清理。该实施例说明 AppFS 的文件协议不仅服务模型回合，也能服务 UI 管理端和桌面启动器，使管理端、`runtime` 和 agent 共享同一生命周期真相源。

（17）技术效果

- 通过将资源、动作和事件统一映射为文件树，agent 可使用普通文件读写完成应用交互，减少针对每个应用硬编码工具/API 的集成成本。
- 通过 ``act`` 追加式动作槽、`action cursor` 和稳定 `client token`，为动作执行提供可审计、可重放、可去重的记录边界，降低重复执行和状态错乱风险。
- 通过 `runtime manifest` 和自描述 ``_app/* .res.json`` 文件，agent 能够根据当前挂载环境动态发现应用能力，减少 `prompt` 中静态塞入大量应用说明的需求。

- 通过 `principal` 与 `attach lease` 分离，使稳定 `agent` 身份不依赖一次性进程；`agent` 重启或恢复会话后仍可复用自己的私有应用实例和 `profile`。
- 通过 ``/public`` 与 ``/private/<principal>`` 命名空间以及可见应用过滤，支持同一项目下多个 `agent` 共享公共资源并隔离私有应用、凭据和事件。
- 通过 `ConnectorContext` 提供有效身份，并把凭据保存在 `connector` 私有状态，避免模型通过 `payload` 冒充其他 `profile`，也避免令牌泄露到模型可见文件或事件。
- 通过事件流和 `InputRouter` 分类策略，将外部应用事件按优先级注入模型回合，实现用户消息、动作失败、凭据状态等事件驱动的 `agent` 响应。
- 通过结构快照与刷新机制，联系人、群聊、`scope` 等动态应用结构能被物化为新的模型可见路径，提升应用状态变化后的可操作性。
- 通过 `Dashboard/desktop` 实施例，操作者可在本地可视化管理多个 `principal` 和 `headless agent`，降低多 `agent` 运行、停止、恢复和清理的操作复杂度。

三、上述技术方案是否有替代方案

有。以下替代方案均可在不脱离本专利构思的情况下实现相同或相近的技术目的，建议代理人在撰写权利要求时酌情概括，避免不必要地限缩到当前项目中的文件名或具体实现。

替代对象	可替代实现
文件系统承载方式	可采用 FUSE、WinFSP、NFS、本地目录、远程挂载、SQLite 虚拟文件系统、对象存储映射或浏览器沙箱文件系统。
资源和动作格式	资源文件可采用 JSON、JSONL、YAML、CBOR、Protocol Buffers 或数据库记录；动作槽可为 <code>`act`</code> 文件、追加式日志、命名管道或事件表。
事件传递方式	事件可写入 <code>`evt.jsonl`</code> ，也可通过 SSE、WebSocket、消息队列、数据库变更流或本地通知通道同步给 <code>agent</code> 。
Connector 部署方式	Connector 可进程内运行，也可通过 HTTP、gRPC、MCP、插件进程、本地子进程或远程服务实现。
身份模型	<code>principal</code> 可表示 <code>agent</code> 、用户、团队、角色或任务执行者； <code>attach lease</code> 可扩展为多活、租约续期、抢占、只读 <code>attach</code> 或审批 <code>attach</code> 。
凭据存储	可使用 <code>connector</code> 内存、加密本地数据库、系统密钥库、远端 vault 或专用 <code>credential service</code> ，但不进入模型可见文件树。
私有 app 策略	<code>path_template</code> 和 <code>profile_template</code> 可由 <code>compose</code> 文件、 <code>registry</code> 、管理端配置、策略引擎或组织权限系统生成。
事件渲染策略	可由 <code>`_app/events.res.json`</code> 、管理端策略、 <code>agent</code> 内置模板或用户配置决定哪些事件唤醒模型、哪些只做上下文摘要。
管理端形态	可为 Web Dashboard、Electron 桌面程序、CLI、IDE 插件或自动化服务，只要通过同一控制动作和 <code>registry</code> 交互即可。

四、本发明的关键技术点

以下按重要性列出建议重点保护的技术手段。该部分不是正式权利要求书，但可供代理人据此撰写方法、系统、电子设备和计算机可读存储介质等不同类型权利要求。

1. 一种将应用结构、资源、动作入口和事件流映射为文件系统树的方法，其中动作入口为追加式动作槽，资源为单对象或多行资源，事件为追加式事件流。
2. 一种 runtime manifest 发现机制，用于向智能体公布 mount root、runtime_session_id、控制动作路径、registry 路径和能力信息。
3. 一种 connector 结构同步机制，其中 connector 返回结构快照，runtime 校验并物化文件树，connector 不直接写 runtime 文件树。
4. 一种基于 action cursor 的动作消费机制，通过 cursor offset 读取新增动作行，检测覆盖和截断，成功后推进 cursor，失败或 transient error 时保留重试能力。
5. 一种动作幂等机制，通过显式 client_token 或由 app_id、session_id、路径、offset、payload 派生的稳定 token 识别重复动作。
6. 一种动作结果事件化机制，将 action.accepted、action.progress、action.completed、action.failed 写入事件流，并与 request_id、client_token 关联。
7. 一种面向智能体的外部事件输入方法，将 connector inbound events 写入事件流，再由 agent 根据事件 cursor 渲染为下一模型回合输入。
8. 一种多 agent principal 身份模型，将 runtime_session_id、attach_id、principal_id 和 profile_id 分层，以区分 runtime、进程、语义身份和应用账号。
9. 一种 private app 自动实例化机制，根据 private app policy 的 path_template 和 profile_template，为每个 principal 自动创建私有应用实例。
10. 一种 attach lease 生命周期机制，通过 attach、heartbeat、status update、stale sweep、detach、delete 管理 agent 在线状态。
11. 一种凭据隔离机制，connector 根据 runtime 提供的 profile_id 使用私有凭据执行动作，凭据不写入模型可见文件树。
12. 一种应用自描述机制，通过 `\_app/actions.res.json`、`\_app/control.res.json`、`\_app/events.res.json`、`\_app/skill.res.json` 向 agent 说明可用动作、事件渲染、使用场景和约束。
13. 一种管理端实施方法，Dashboard 或桌面 launcher 通过读取 registry 和追加 principal 动作管理 agent 生命周期，并向 agent 进程注入 AppFS 环境变量。
14. 一种基于 Tinode 的私有聊天实施例，其中每个 principal 对应独立 Tinode profile，动作文件用于发送消息或创建群，inbox 资源用于读取消息，事件流用于唤醒 agent。

五、建议附图说明

附图编号	建议名称	绘制要点
图 1	系统总体架构图	展示 agent、AppFS mount、runtime、connector、registry、event stream、Dashboard/desktop 的关系。
图 2	文件树命名空间图	展示 /.well-known/appfs/runtime.json、/_appfs、/public、/private、_app、_stream、.res/.act/.evt 文件。
图 3	动作处理流程图	展示向 .act 追加、consumer 读取、cursor、ConnectorContext、connector 执行、事件写入。
图 4	事件唤醒流程图	展示 connector inbound events、runtime event stream、agent event cursor、pending input、model call。
图 5	principal 生命周期图	展示 create、attach、heartbeat、status update、stale sweep、detach、delete 和 private app materialization。
图 6	Tinode 私有聊天实施例图	展示两个 principal 分别具有 /private/default/tinode 和 /private/code-implementer/tinode，并通过 Tinode connector 发送 direct/group message。

六、其他有助于理解本申请提案的技术资料

以下资料来自当前 appfs-platform 项目，可供代理人理解实施例、接口命名和技术细节。正式申请文件中可根据需要将内部文件路径改写为一般性技术描述。

资料	对应内容
appfs/sdk/rust/src/appfs_connector.rs	ConnectorContext、AppConnector、SubmitActionRequest、ConnectorInboundEvent、AppStructureNode 等抽象。
appfs/cli/src/cmd/appfs/runtime_manifest.rs	`./.well-known/appfs/runtime.json` 生成和控制平面路径。
appfs/cli/src/cmd/appfs/tree_sync.rs	结构同步、节点物化、runtime scaffolding、events.evt.jsonl、action-cursors.res.json。
appfs/cli/src/cmd/appfs/action_consumer.rs	`/.act` cursor、JSONL 读取、稳定 client_token、覆盖/截断检测、多行恢复。
appfs/cli/src/cmd/appfs/events.rs	事件字段、 action.completed/action.failed/action.accepted/action.progress 发布。
appfs/cli/src/cmd/appfs/registry.rs	apps registry、app policy registry、principal registry、status view、stale 阈值。

资料	对应内容
appfs/cli/src/cmd/appfs/runtime_supervisor.rs	principal create/update/delete/attach/detach、private app materialization、stale sweep。
appfs-agent/rust/crates/runtime/src/appfs.rs	AppFS 环境发现、principal ensure、attach、heartbeat、AppFS prompt、事件提醒渲染。
appfs-agent/rust/crates/runtime/src/conversation.rs	模型调用前收集 AppFS pending inputs 和事件 cursor 更新。
appfs/sdk/rust/src/tinode_connector.rs	Tinode 私有应用结构、凭据、发送消息、群组、inbox、inbound events。
dashboard/server/src/principal-lifecycle.ts	Dashboard 读取 principal 视图、追加 principal 动作、创建/删除/恢复 principal。
dashboard/server/src/process-manager.ts	Dashboard 启动 agent 并注入 AppFS principal、attach、mount、manifest 环境变量。
desktop/src/server-launcher.ts	Electron 桌面启动 dashboard server 和相关二进制环境。
docs/APPFS-multi-agent-identity-and-app-visibility-v0-design.md	multi-agent identity 与 private app visibility 设计。
docs/TINODE-APPFS-v0-design.md 和 docs/TINODE-APPFS-tree-v0-design.md	Tinode 实施例设计。
integration/APPFS-appfs-agent-attach-contract-v1.1.md	attach contract 与 launcher contract。

七、待代理人确认或可补充内容

- 是否将 Tinode 放入主权利要求，还是仅作为说明书实施例。建议主权利要求保护通用 AppFS 机制，Tinode 放实施例。
- 是否将 Dashboard/desktop launcher 放入从属权利要求。建议作为管理端实施方式展开。
- 是否单独拆分第二件专利：多 agent principal 身份和私有应用实例化机制。当前版本将其作为核心方案的一部分。
- 是否单独拆分第三件专利：追加式`.act`动作槽与事件流驱动模型回合。该点也具备独立保护价值。
- 正式申请文件需要代理人进一步撰写权利要求书、说明书、摘要和附图，本交底书仅作为技术材料。